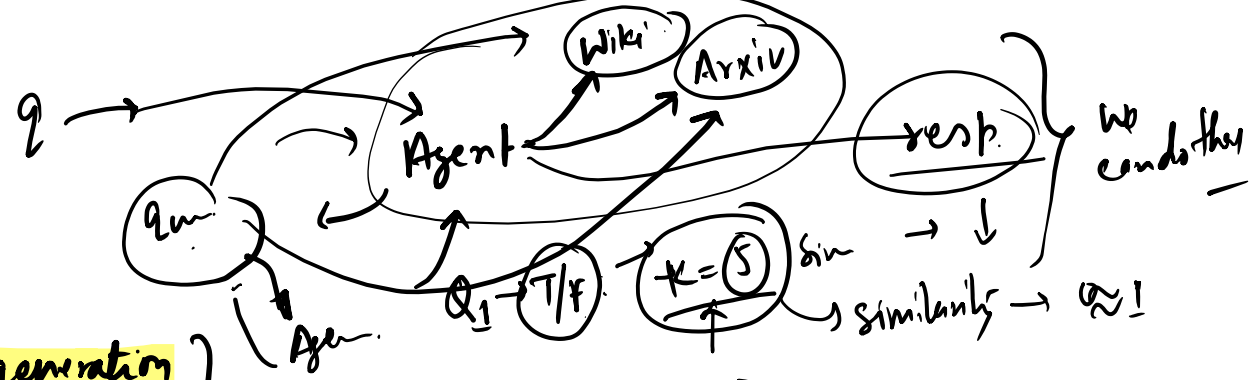


Agentic AI

What is an AI Agent?

LangGraph



CA ⇒ chatbots. ⇒ very intelligent ⇒ reasoning + generation

Knowledge cutoff

GenAI application.



Query

LLM

response

gemma2:9b

{ what is the latest IP News }

what is the current temp of Noida ~ Delhi NCR

Connect this LLM + external tools

LLM = Llama 3.1 70B
LLM bind tools multiply (tools)

Input

LLM

tools

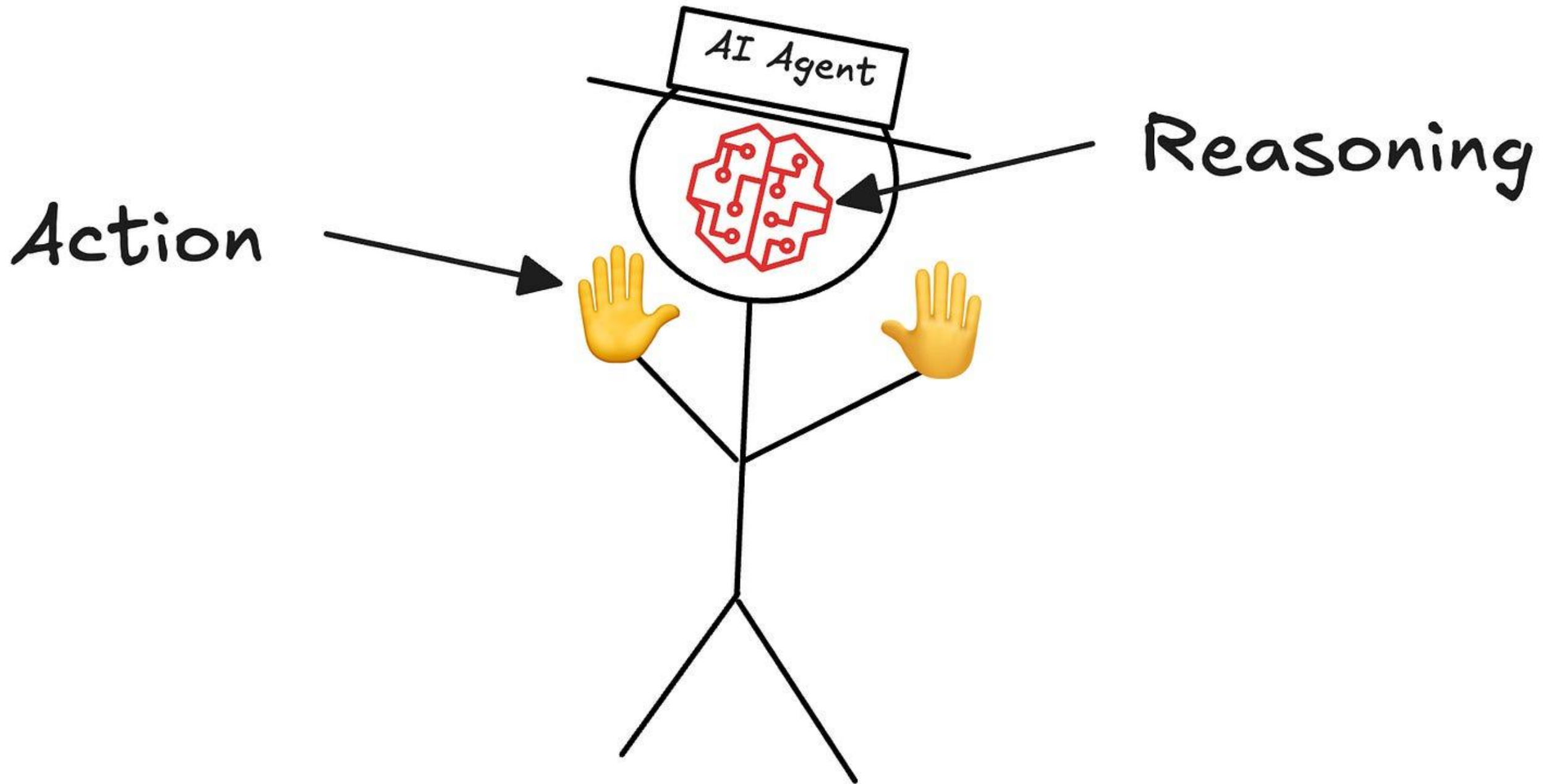
Tools

response

Today's Temp.

Agent

- LLM ⇒ Brain
- Tool calls
- Memory
- Autonomy

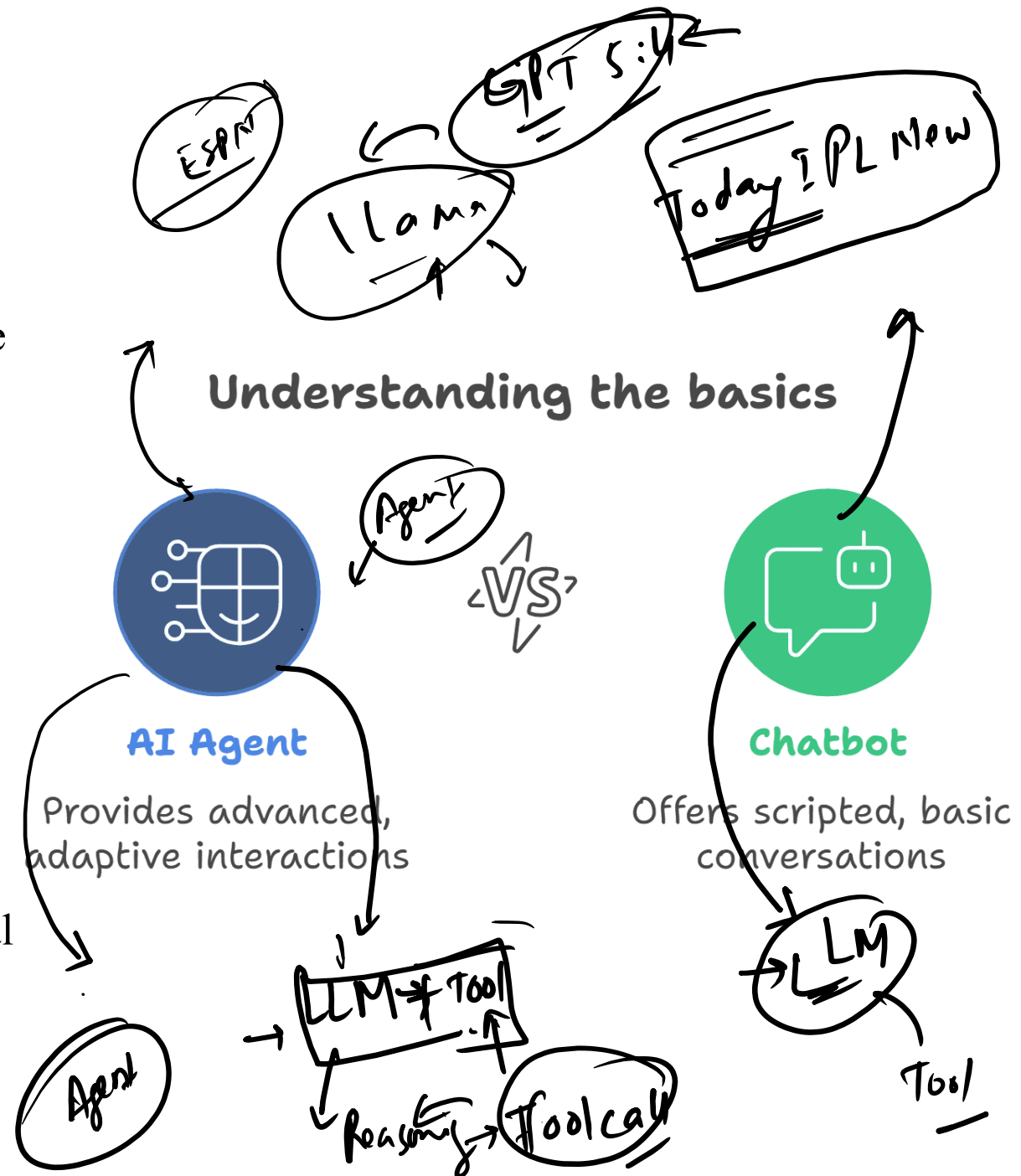


What is an AI Agent?

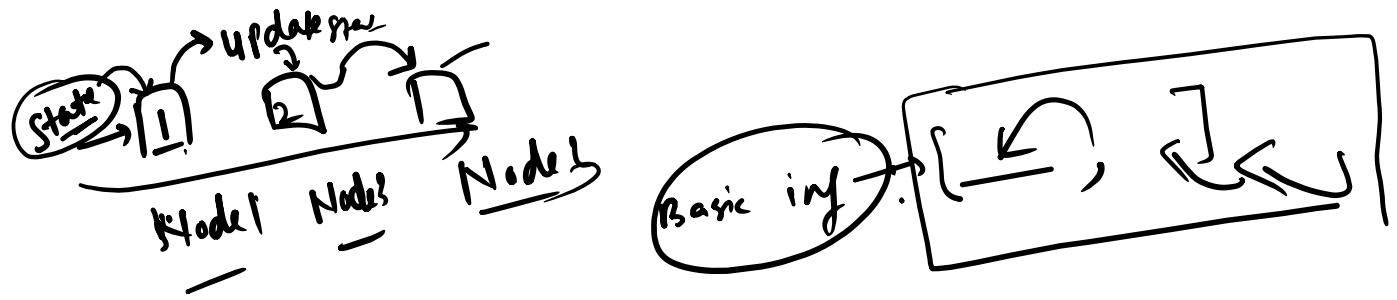
- An AI agent is a software-based system that is able to perceive information, reason over that information, and take action to achieve a defined goal.

Reasoning + Action – ReACT Agents

- Key characteristics-
 - ✓ LLM as Brain – can do reasoning over the information
 - Tool call – can call external tools
 - Must be goal oriented
 - Memory to track the progress across long task.
 - Autonomous – take decisions on its own.
- AI agents are single entity with focused purpose. They make decisions autonomously based on a real time data and circumstances to achieve a goal.



LangGraph



- **LangChain + LCEL:** LangChain provides a framework for building LLM applications, with LCEL simplifying step-by-step action sequences as directed acyclic graphs (DAGs).
- **LLMs as reasoning engines:** Modern agents use LLMs not only for execution but also for reasoning, introducing cycles and complex conditions.
- **LangGraph module:** To handle these advanced scenarios beyond LCEL's DAG limitations, LangChain introduced LangGraph, enabling richer agent workflows with loops and conditional logic.
- LangGraph models all the interactions as **cyclical graphs**, enabling advanced **agent** and **multi-agent workflows** with **loops** and **conditional logic**.
- LangGraph is part of the LangChain ecosystem. LangGraph is created to define cyclical graphs. Graphs consist of the following elements:
 - **Nodes** represent actual actions and can be either LLMs, agents or functions. Also, a special END node marks the end of execution.
 - **Edges** connect nodes and determine the execution flow of your graph. There are **basic edges** that simply link one node to another and **conditional edges** that incorporate if-statements and additional logic.
 - **State** – State Schema serves as the input function for all nodes. Basically

AI in message

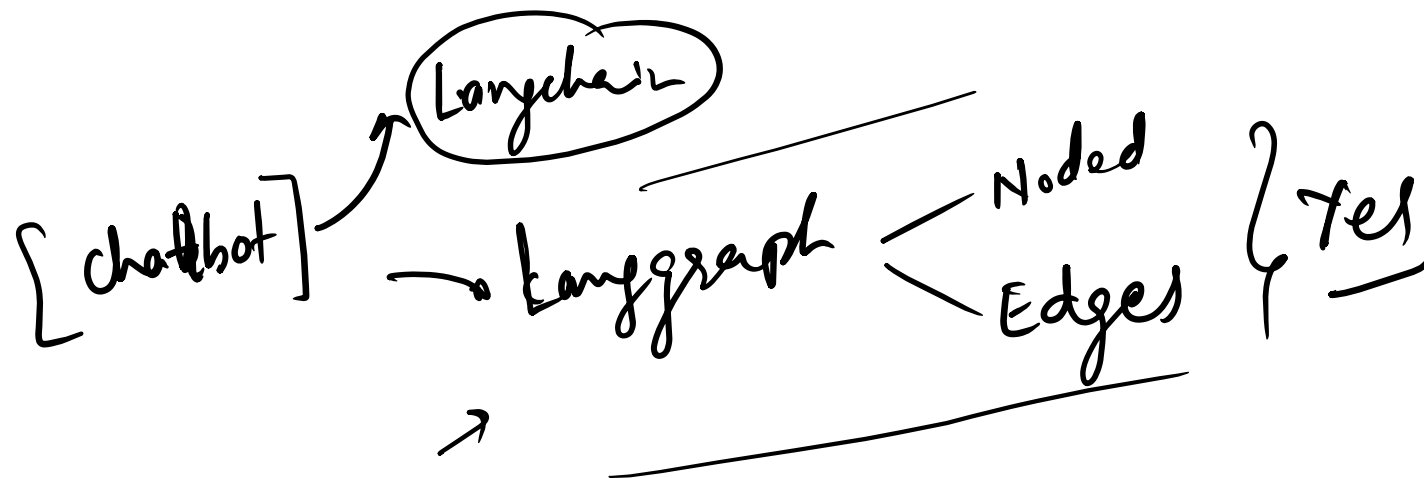
reduce

Edge - N - N

Condition

LangGraph

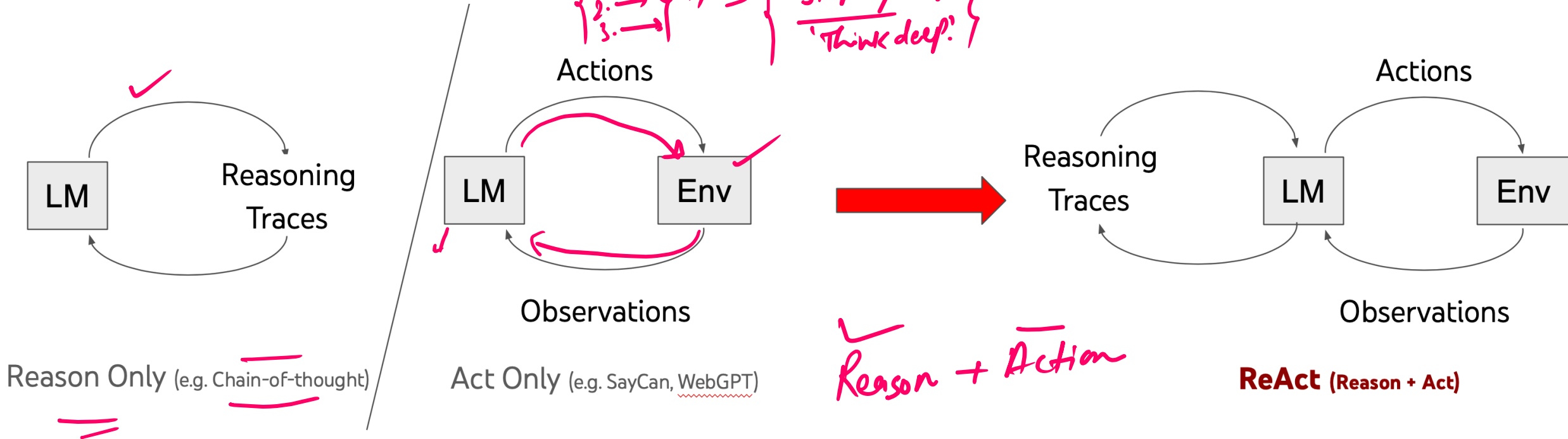
- **LangChain + LCEL:** LangChain provides a framework for building LLM applications, with LCEL simplifying step-by-step action sequences as directed acyclic graphs (DAGs).
- **LLMs as reasoning engines:** Modern agents use LLMs not only for execution but also for reasoning, introducing cycles and complex conditions.
- **LangGraph module:** To handle these advanced scenarios beyond LCEL's DAG limitations, LangChain introduced LangGraph, enabling richer agent workflows with loops and conditional logic.
- LangGraph models all the interactions as **cyclical graphs**, enabling advanced **agent** and **multi-agent workflows** with **loops** and **conditional logic**.
- LangGraph is part of the LangChain ecosystem. LangGraph is created to define cyclical graphs. Graphs consist of the following elements:
 - **Nodes** represent actual actions and can be either LLMs, agents or functions. Also, a special END node marks the end of execution.
 - **Edges** connect nodes and determine the execution flow of your graph. There are **basic edges** that simply link one node to another and **conditional edges** that incorporate if-statements and additional logic.
 - **State** – State Schema serves as the input function for all nodes. State holds all the information the agent knows at any point in time. Every node reads from it and writes back to it. Think of it as a shared whiteboard.



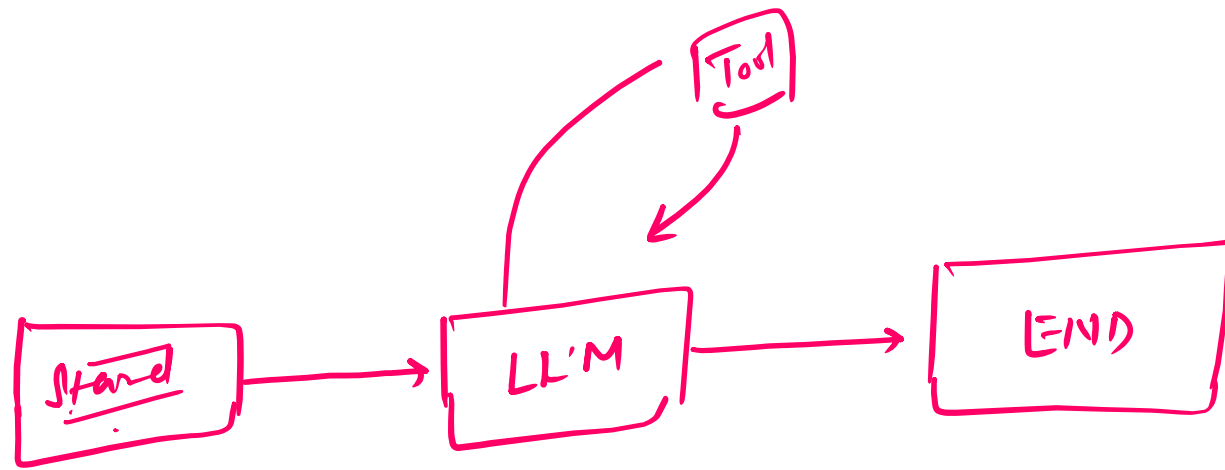
Graph

✓ ReACT Agents

actions
Book the flight ticket
LLM → very good at reasoning
→ COT ⇒ Reasoning ⇒ Think deeper
1. →
2. →
3. →
⇒ Tools
'step-by-step'
'Think deep!'



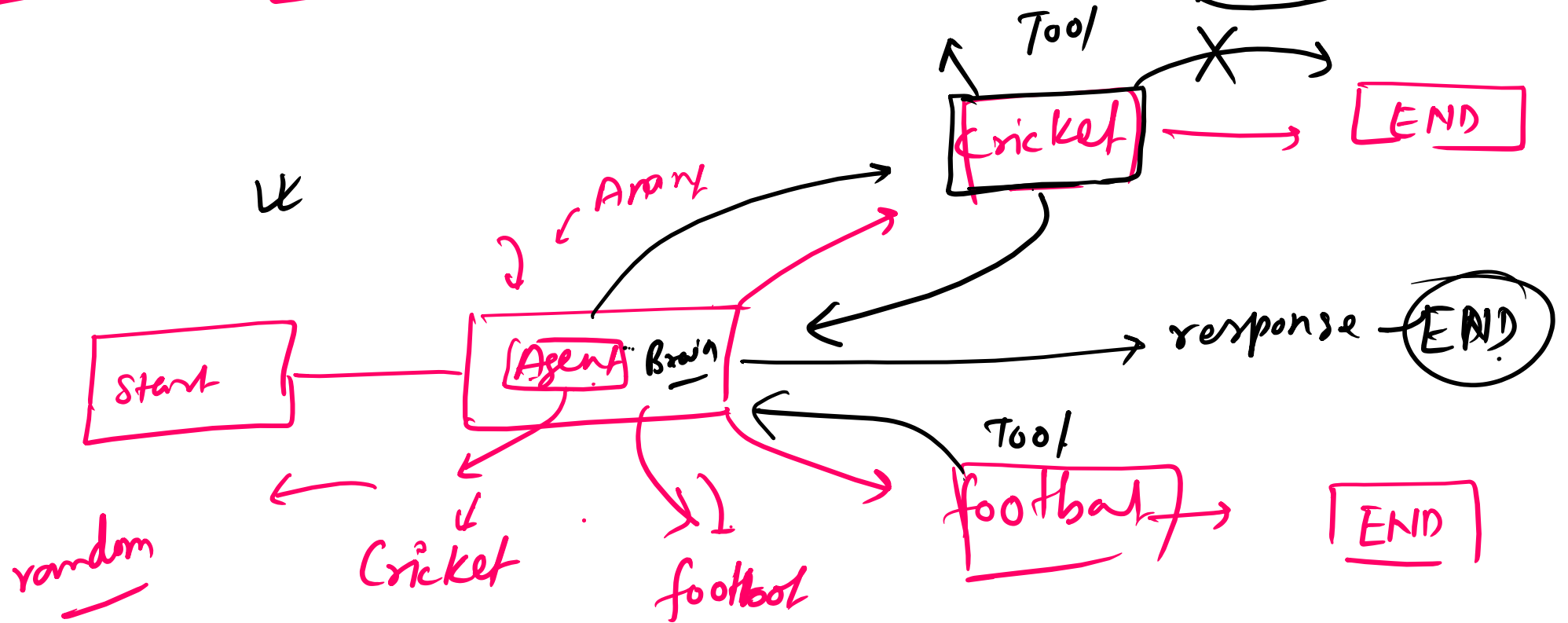
LLMs have traditionally treated reasoning (thinking through a problem) and acting (executing steps or using tools) as separate abilities, but frameworks like ReAct now combine them, enabling models to interleave thought and action in dynamic tasks. This shift makes LLMs more adaptive, allowing them to reason, act, and adjust in real time.



{ Reason → Act → observe → END }

→ Simple chat bot

ReAct



Reasoning vs Acting in LLMs

1. Reasoning (Chain-of-Thought Prompting)

• **Definition:** The model generates intermediate steps to logically arrive at an answer.

• **Example:**

- **Math Problem:** “What is 23×47 ?”

- Reasoning:

- Breaks into steps $\rightarrow 23 \times 40 = 920$,

- $23 \times 7 = 161$,

- then adds $\rightarrow 920 + 161 = \underline{1081}$.

- Output: “1081.”

• **Use Case:** Solving puzzles, logical deduction, multi-step questions.

few shot
zero shot

Reason $\rightarrow$ CoT $\rightarrow$ Break

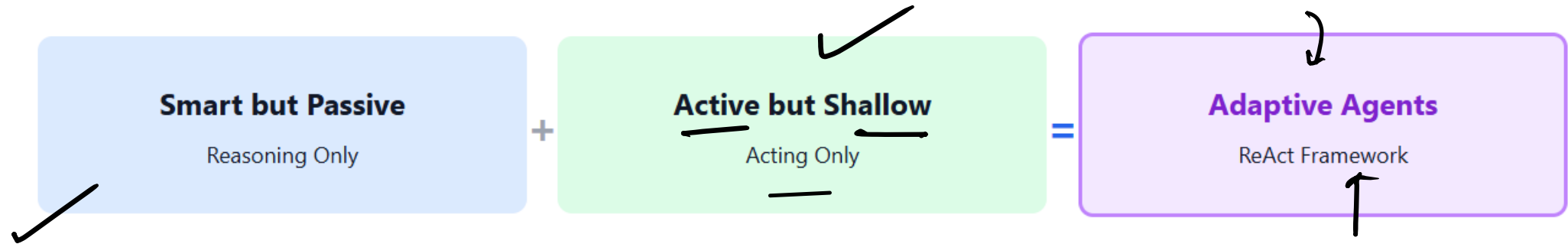
- **Acting (Action Plan Generation)** ✓
- **Definition:** The model generates executable steps or commands to interact with the world.
- **Example:**
 - **Task:** “Book a flight from Delhi to Mumbai.”
 - Acting: _____
 - Produces an action plan →
 - Step 1: Search flight options. ✓
 - Step 2: Compare prices. Step 3: Reserve ticket ✓
 - If connected to APIs, the agent executes these steps.
- **Use Case:** Robotics, customer service bots, workflow automation.

IRC
Disha ⇒

↓
Maze Solver

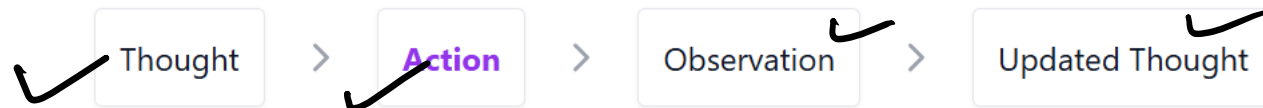
Handwritten signature or initials inside a box.

The Integration Gap – Reasoning + Act



ReAct (Reason + Act)

A framework where the model interleaves thought traces (Reasoning) and task-specific actions (Acting) in an iterative loop.



How ReAct works?

Just like a human solving a problem:

THINK

Assess what is known vs. unknown.

ACT

Gather data or execute a tool.

OBSERVE

Process the result of the action.

ADAPT

Refine the plan based on new info.

Question: Who is the US president? Find a recent article about him.

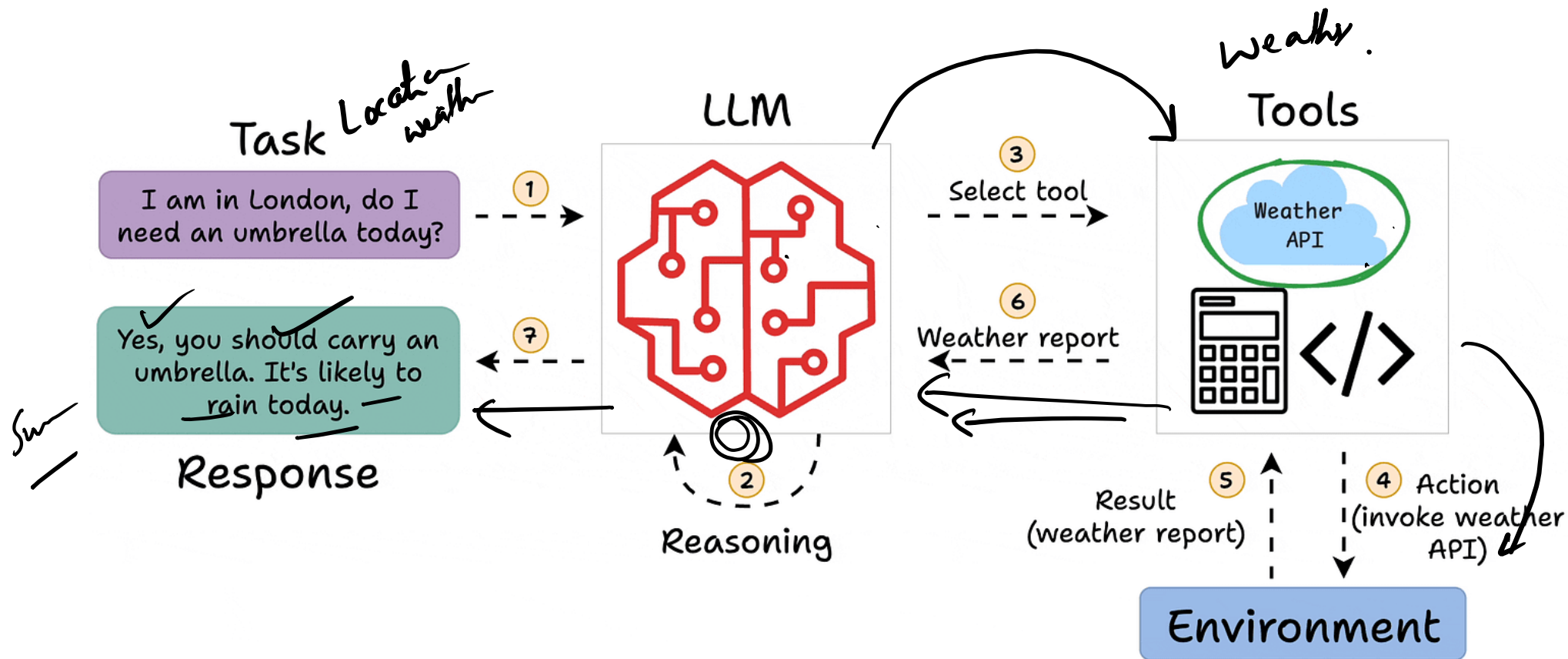
Thought: I need to identify the current president first. I will use a search tool.

Action: `search("current US president")`

Observation: Donald Trump was inaugurated on Jan 20, 2025.

Thought: The president is Donald Trump. Now I need a news article about his recent activity.

Action: `search("Donald Trump news latest")`



- Understand the query (language understanding)
- Select the tools it would need to generate a response (the weather API).
- Gather the weather report.
- Understand if it's going to rain and produce an answer accordingly.

ReACT Agent

Comparison Table

Aspect	Reasoning (CoT)	Acting (Plans)	ReAct (Combined)
Focus ✓	Logical steps ✓	Executable actions ✓	<u>Interleaved both</u> ✓
Strength	Logical Accuracy ✓	Interaction ✓	Adaptive Decisioning
Weakness	No tool use ✓	Shallow logic ✓ ←	Complex design ✓

LangGraph

num : ~~int~~ list [int, 'inter : - - - -]

